

Proyecto

Implementación de una Plataforma Web Resiliente basada en WordPress

Estudiante:

Romero Luzardo Alan

Asignatura:

Administración de servidores

Docente:

Ing. Manzano Araujo Renato Javier

Instituto Superior Tecnológico Juan Bautista Aguirre

Fecha:

08 de Enero del 2026

1. Introducción

En la actualidad, la disponibilidad de los servicios web es un factor crítico para garantizar una correcta experiencia al usuario. La caída de un servidor puede provocar interrupciones que afecten la continuidad del servicio y la integridad de la información. Por esta razón, surge la necesidad de implementar arquitecturas resilientes que permitan mantener los sistemas operativos aun cuando uno de sus componentes falle.

El presente informe describe el diseño e implementación de una **plataforma web resiliente utilizando WordPress**, apoyada en servidores redundantes, almacenamiento compartido y replicación de base de datos. El proyecto fue desarrollado en un entorno académico, con el objetivo de simular escenarios reales de alta disponibilidad y tolerancia a fallos.

2. Objetivo del proyecto

2.1 Objetivo general

Implementar una plataforma WordPress con alta disponibilidad que garantice la continuidad del servicio web ante la falla de uno de los servidores.

2.2 Objetivos específicos

- Desplegar WordPress en dos servidores web independientes para asegurar redundancia.
- Configurar un sistema de replicación de base de datos que reduzca el riesgo de pérdida de información.
- Implementar almacenamiento compartido para mantener la coherencia de los archivos del sitio.
- Validar el funcionamiento del sistema ante la caída de un servidor.
- Analizar los tiempos de recuperación y proponer mejoras para optimizar RPO y RTO.

3. Descripción del entorno de trabajo

El proyecto fue desarrollado utilizando máquinas virtuales sobre un equipo con recursos limitados, lo que obligó a tomar decisiones técnicas enfocadas en la eficiencia. Se empleó un sistema operativo orientado a servidores, sin entorno gráfico, ya que la administración del sitio WordPress se realiza completamente desde un navegador web.

La gestión de las máquinas virtuales se llevaron a cabo de forma remota mediante conexiones seguras, permitiendo un control centralizado de todos los servicios involucrados.

4. Arquitectura de la solución

La solución implementada se diseñó bajo un enfoque por capas, permitiendo separar responsabilidades y facilitar la tolerancia a fallos:

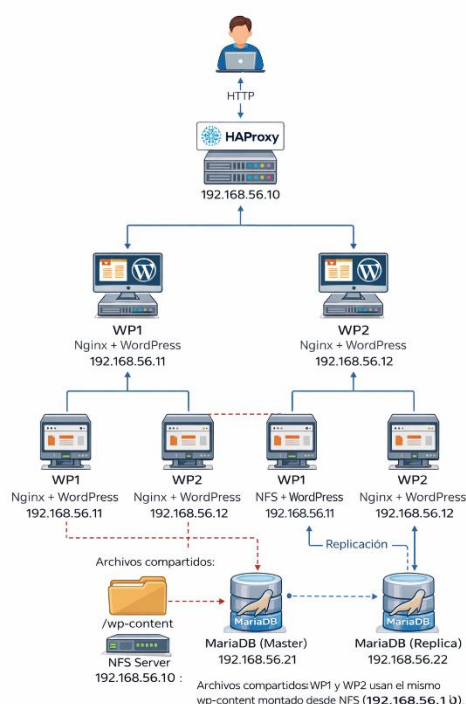
- **Capa de entrada:** encargada de recibir las solicitudes de los usuarios y distribuirlas.
- **Capa web:** conformada por dos servidores WordPress configurados de forma idéntica.
- **Capa de almacenamiento:** responsable de mantener los archivos compartidos del sitio.
- **Capa de datos:** encargada de almacenar y replicar la información del sistema.

Esta arquitectura permite que el sistema continúe funcionando aun cuando uno de los componentes falle.

4.1 Componentes y direccionamiento IP

VM	Rol	IP interna	Red	Notas
LB+NFS	Balanceador + NFS	192.168.56.10	Host-Only + NAT	Punto de entrada del usuario. Exporta wp-content por NFS.
WP1	WordPress #1	192.168.56.11	Host-Only + NAT	Nginx/PHP-FPM/WordPress. Monta wp-content desde NFS.
WP2	WordPress #2	192.168.56.12	Host-Only + NAT	Nginx/PHP-FPM/WordPress. Monta wp-content desde NFS.
DB1	MariaDB Master	192.168.56.21	Host-Only + NAT	Base de datos principal (escrituras).
DB2	MariaDB Réplica	192.168.56.22	Host-Only + NAT	Replica desde DB1 (solo lectura).

4.2 Diagrama Lógico



5. Implementación del sistema

5.1 Servidores web redundantes

Se desplegaron dos servidores WordPress independientes que comparten la misma configuración y acceden a los mismos recursos. Ambos servidores están preparados para atender solicitudes de los usuarios, pero el acceso se realiza siempre a través de un único punto de entrada.

5.2 Almacenamiento compartido

Uno de los principales desafíos de WordPress en entornos redundantes es la gestión de los archivos. Para resolver este problema, se implementó un almacenamiento compartido donde se centralizó la carpeta que contiene temas, plugins y archivos subidos por los usuarios. De esta manera, cualquier cambio realizado en un servidor es inmediatamente visible en el otro.

5.3 Balanceo de carga y tolerancia a fallos

Se configuró un balanceador de carga encargado de distribuir el tráfico entre los dos servidores web disponibles. Este componente monitorea constantemente el estado de los servidores y, en caso de detectar una falla, redirige automáticamente las solicitudes al servidor activo, evitando interrupciones del servicio.

5.4 Replicación de base de datos

La base de datos fue configurada bajo un esquema maestro–réplica. El servidor principal gestiona las operaciones de escritura, mientras que el servidor réplica mantiene una copia actualizada de la información. Esta configuración permite contar con respaldo de los datos y reduce el impacto ante fallos en la base de datos principal.

6. Pruebas realizadas

Para verificar el correcto funcionamiento del sistema, se realizaron diversas pruebas:

- **Pruebas de consistencia de archivos**, comprobando que ambos servidores web acceden al mismo contenido.
- **Pruebas de caída de servidores web**, simulando fallos y verificando que el sitio sigue disponible.
- **Pruebas de replicación de base de datos**, validando que los cambios se reflejan correctamente en el servidor réplica.

Los resultados demostraron que el sistema responde adecuadamente ante fallas comunes.

7. Resultados y análisis

Los resultados obtenidos evidencian que la plataforma implementada cumple con los objetivos planteados. El sitio WordPress se mantiene accesible a través de un único punto de entrada, incluso cuando uno de los servidores web deja de funcionar. Asimismo, el uso de almacenamiento compartido garantiza la coherencia de los archivos, y la replicación de base de datos proporciona una capa adicional de protección de la información.

8. RPO y RTO

El **RPO (Recovery Point Objective)** y el **RTO (Recovery Time Objective)** son métricas fundamentales en planes de recuperación ante desastres y alta disponibilidad. Estas métricas permiten evaluar cuánto tiempo puede estar un servicio fuera de línea y cuánta información podría perderse ante una falla.

En este proyecto se implementó una solución basada en **WordPress redundante, balanceo de carga con HAProxy, replicación de base de datos MariaDB y almacenamiento compartido mediante NFS**. Aunque la arquitectura garantiza alta disponibilidad en la capa web, existen oportunidades de mejora en los valores de RPO y RTO, especialmente en la capa de datos.

9. Evaluación del RPO y RTO en la implementación actual

9.1 RPO (Recovery Point Objective)

En la implementación actual, la base de datos MariaDB utiliza un esquema de replicación **maestro-réplica de tipo asíncrono**. Esto implica que los cambios realizados en el servidor principal (DB1) se replican al servidor secundario (DB2) con un pequeño retraso.

Debido a esta característica:

- Si el servidor DB1 falla antes de que los cambios se hayan replicado completamente a DB2, existe la posibilidad de perder las últimas transacciones.
- El **RPO actual no es cero**, y puede variar dependiendo de la carga del sistema y la frecuencia de escritura en la base de datos.
- En un escenario académico como este, el RPO puede estimarse en **segundos o pocos minutos**.

9.2 RTO (Recovery Time Objective) En

cuanto al tiempo de recuperación del servicio:

- **RTO en la capa web:**
Gracias al uso de HAProxy, si uno de los servidores WordPress (WP1 o WP2) deja de funcionar, el balanceador redirige automáticamente el tráfico al servidor disponible. Esto permite un **RTO cercano a cero**, ya que el usuario no percibe interrupciones.
- **RTO en la base de datos:**
Actualmente, el sistema no cuenta con un mecanismo de failover automático para la base de datos. En caso de falla del servidor principal (DB1), la promoción de DB2 a servidor master debe realizarse de forma manual por el administrador.
Esto incrementa el **RTO de la capa de datos**, ya que el proceso de recuperación podría tomar varios minutos.

10. Propuestas de mejora para optimizar RPO y RTO

10.1 Mejoras para reducir el RTO

1. Alta disponibilidad del balanceador de carga

Actualmente, HAProxy se ejecuta en una sola máquina virtual, lo que representa un punto único de falla. Como mejora, se propone implementar un segundo balanceador junto con un IP virtual compartido mediante herramientas como Keepalived.

Ventaja: Si el balanceador principal falla, el secundario asume el control automáticamente, manteniendo el RTO de la capa web prácticamente en cero.

2. Automatización del failover de base de datos

El uso de herramientas como ProxySQL u Orchestrator permitiría automatizar la promoción del servidor réplica (DB2) a master en caso de fallo de DB1.

Ventaja: Se elimina la intervención manual y se reduce considerablemente el RTO de la base de datos.

3. Sistema de monitoreo y alertas

La implementación de herramientas de monitoreo como Zabbix, Prometheus o Nagios permitiría detectar fallos en tiempo real en servicios como WordPress, NFS, MariaDB y HAProxy.

Ventaja: Una detección temprana reduce el tiempo de respuesta y, por tanto, el RTO.

10.2 Mejoras para reducir el RPO

1. Replicación síncrona o clúster de base de datos

El uso de soluciones como **MariaDB Galera Cluster** permitiría replicación síncrona entre nodos, garantizando que los datos se escriban simultáneamente en todos los servidores.

Ventaja: El RPO sería cero, ya que no existiría pérdida de datos ante la caída del servidor principal.

2. Backups frecuentes y restauración a punto en el tiempo

La implementación de copias de seguridad incrementales junto con el uso de binlogs permitiría restaurar la base de datos a un punto específico en el tiempo.

Ventaja: Se reduce significativamente el RPO incluso en escenarios de fallas graves.

3. Replicación semisincrónica

Como alternativa intermedia, se puede implementar replicación semisincrónica, donde al menos una réplica confirme la recepción de los datos antes de completar la transacción.

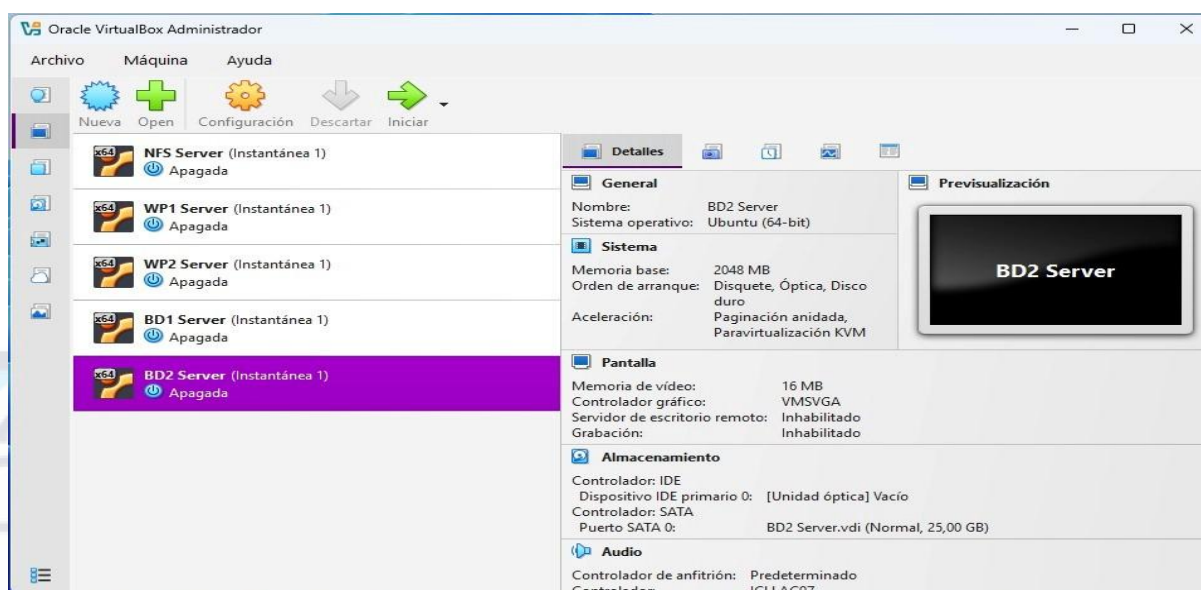
Ventaja: Reduce el RPO sin el costo total de una replicación completamente síncrona.

11. Conclusiones finales

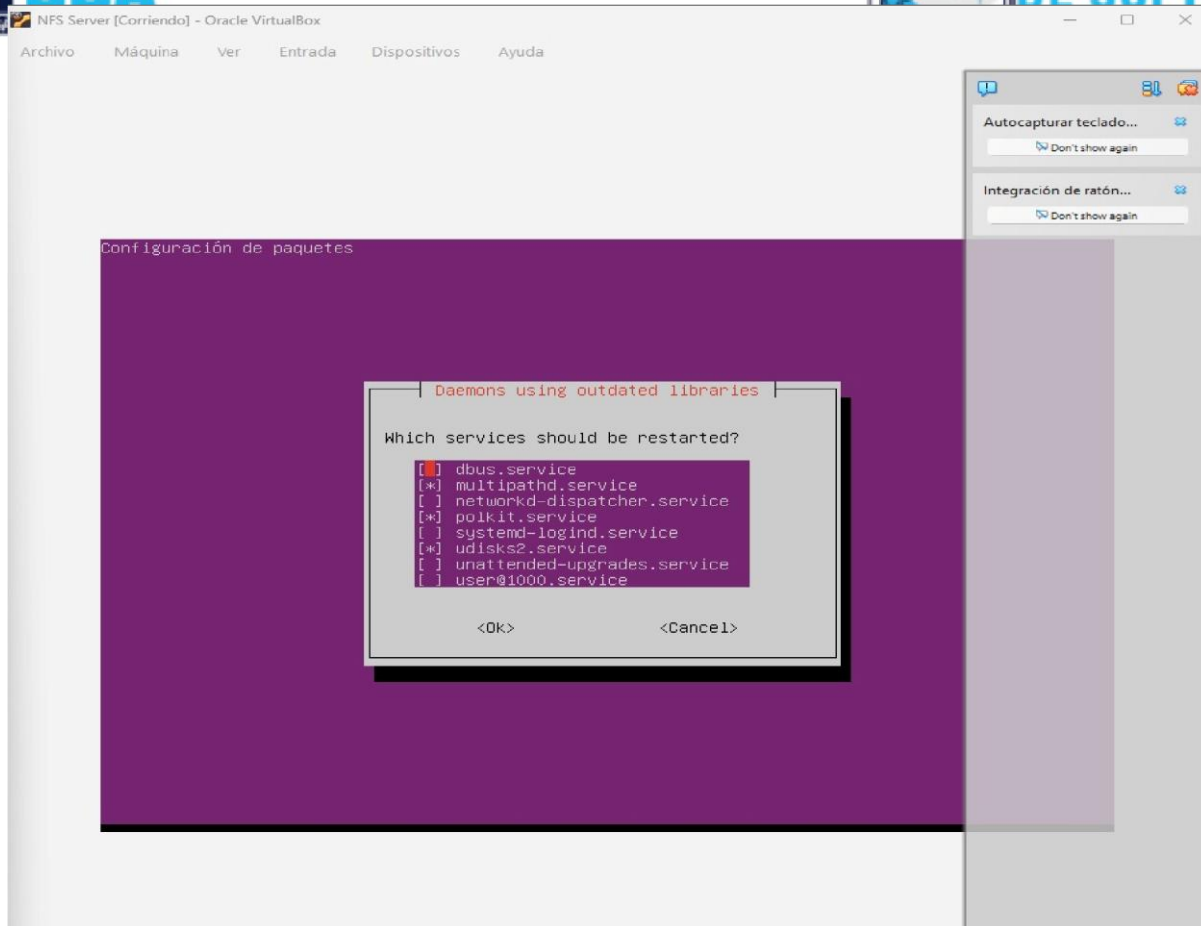
La evaluación del RPO y RTO demuestra que la plataforma implementada ofrece **alta disponibilidad en la capa web**, con tiempos de recuperación prácticamente inmediatos. Sin embargo, en la capa de datos aún existen oportunidades de mejora, principalmente relacionadas con la automatización del failover y la reducción de la posible pérdida de información.

Las propuestas presentadas permiten proyectar esta solución académica hacia un entorno más cercano a producción, fortaleciendo la resiliencia del sistema y garantizando una recuperación rápida y segura ante fallos.

12. Capturas de pantalla



Captura 1. Acceso a la plataforma WordPress



Captura 2. Panel de administración de WordPress

```
bd1server@bd1server:~$ sudo mysql -e "SHOW MASTER STATUS\G"
***** 1. row *****
      File: mysql-bin.000004
      Position: 1453
      Binlog_Do_DB:
      Binlog_Ignore_DB:
bd1server@bd1server:~$
```

Captura 3. Funcionamiento del balanceador de carga

```
bd2server@bd2server:~$ sudo mysql -e 'CHANGE MASTER TO MASTER_HOST="192.168.56.21", MASTER_USER="rep
1", MASTER_PASSWORD="ReplPass123!", MASTER_LOG_FILE="mysql-bin.000004", MASTER_LOG_POS=1453;'
>
```

Captura 4. Almacenamiento compartido del sistema

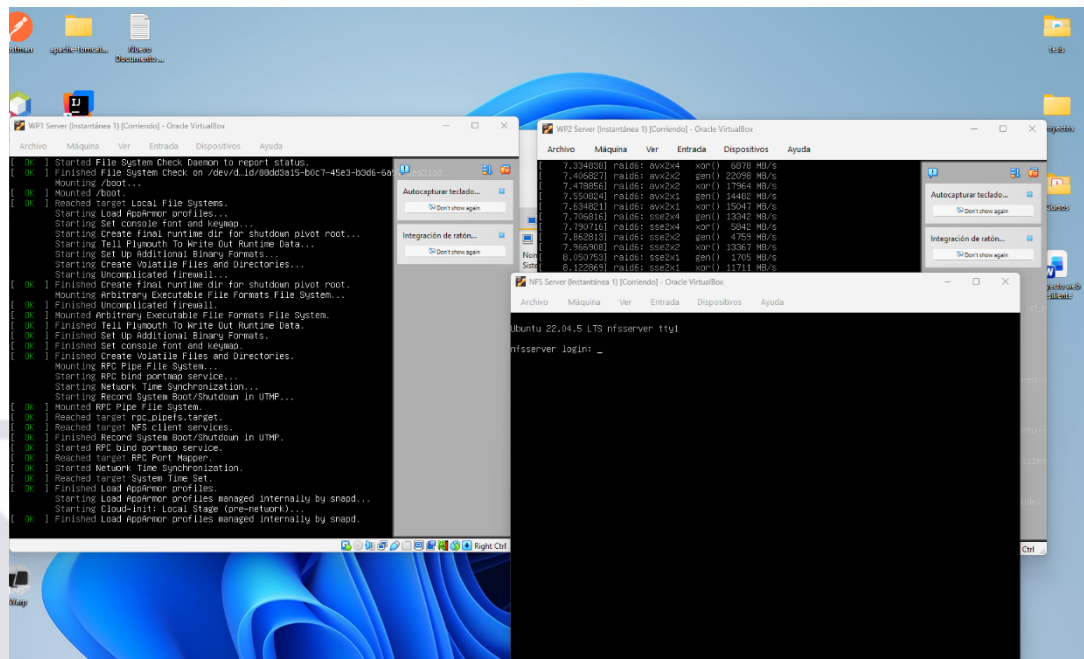
```
GNU nano 6.2 /etc/haproxy/haproxy.cfg
log /dev/log local1 notice
chroot /var/lib/haproxy
stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd listeners
stats timeout 30s
user haproxy
group haproxy
daemon

# Default SSL material locations
ca-base /etc/ssl/certs
crt-base /etc/ssl/private

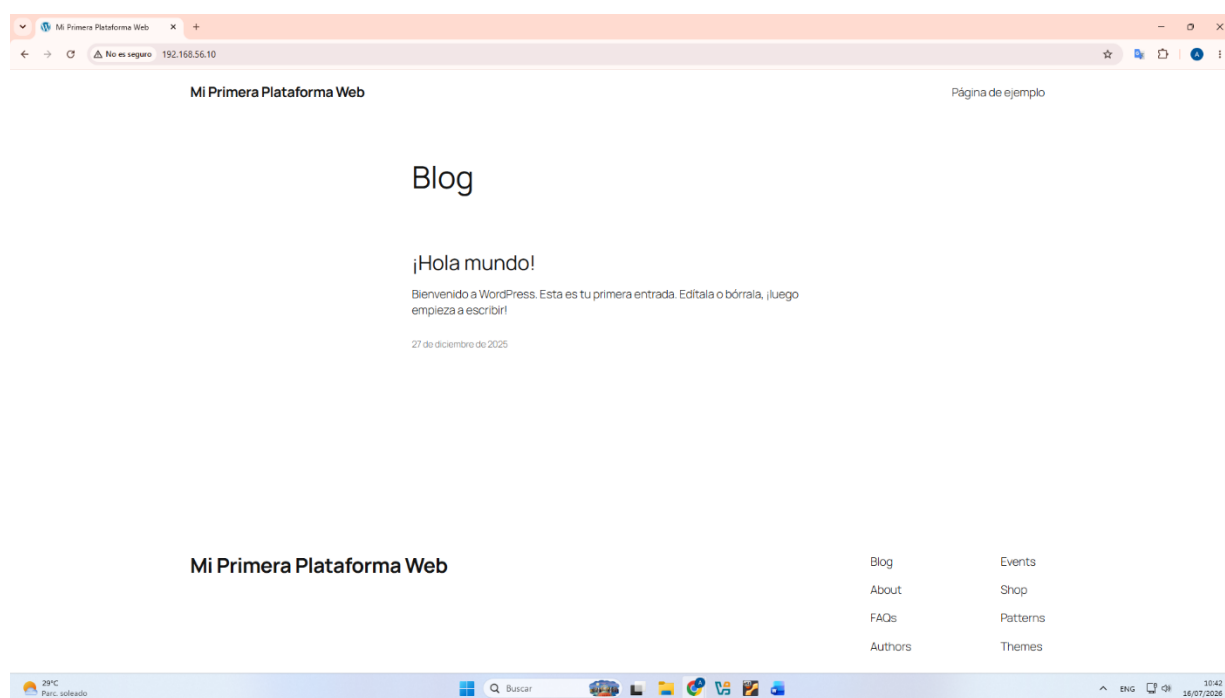
# See: https://ssl-config.mozilla.org/#server=haproxy&server-version=2.0.3&config=intermediate
ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-EC-
ssl-default-bind-ciphersuites TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POL
ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
    log global
    mode http
    option httplog
    option dontlognull
    timeout connect 5000
    timeout client 50000
    timeout server 50000
    errorfile 400 /etc/haproxy/errors/400.http
    errorfile 403 /etc/haproxy/errors/403.http
    errorfile 408 /etc/haproxy/errors/408.http
    errorfile 500 /etc/haproxy/errors/500.http
    errorfile 502 /etc/haproxy/errors/502.http
    errorfile 503 /etc/haproxy/errors/503.http
    errorfile 504 /etc/haproxy/errors/504.http
```

Captura 5. Replicación de la base de datos



Captura 6. Iniciando las VM



Captura 7. Resultado del proyecto